第 07 章

业务状态、检查点与幂等恢复

解释为什么最新图快照不一定可以加入对话，以及重复请求如何避免重复执行

这一章完成后，你能够

- 区分 PostgreSQL 业务事实、Redis 业务快照和 LangGraph 检查点。
- 跟踪消息 ID、轮次 ID、图线程 ID 和接受检查点的对应关系。
- 理解会话锁、请求摘要、短事务和失败轮次历史过滤。
- 通过离线图实验观察未接受状态隔离，明确它与真实 PostgreSQL 验收的证据边界。

学习前提

第05章 Agent 工具循环和证据校验；第06章 State、checkpointer 和 thread_id；数据库事务、唯一约束和 async context manager 基础

代码基准：72dea46 + 当前已完成的学习准备。Python 3.12；LangChain 1.4.3；LangGraph 1.2.12。

正文采用场景、名词、步骤、源码、实验和练习的教学顺序。截图注明数据来源；教学简化与生产实现分别标记。

本章目录

7.1 一个容易忽略的问题：失败草稿会不会污染下一轮 3

7.2 四种 ID 与两种状态：先建立对应关系 4

7.3 PostgreSQL Checkpointer：初始化和运行应分开 6

7.4 幂等：同一消息重复发来，应重放什么 7

7.5 执行锁与短事务：等待模型时不持有长业务事务 8

7.6 从接受指针启动：Overwrite 为什么不可少 10

7.7 中断恢复：仅补交完整且重新验证过的结果 12

7.8 Redis Snapshot：快速恢复学习界面，不接管图历史 13

7.9 完整离线实验：最新快照与接受快照不同 15

7.10 综合练习与参考思路 17

7.11 源码阅读清单与下一步 18

7.1 一个容易忽略的问题：失败草稿会不会污染下一轮

假设模型已经检索资料并生成草稿，图也保存了检查点，但服务端发现引用过期，或者进程在提交前退出。图的最新快照可能包含这次未接受输出。用户再次提问时，如果程序直接沿用最新状态，就可能把失败草稿、旧证据和未成功的问题作为可信历史。持久化不只是“保存”，还必须决定“哪些状态已被业务接受”。



图 7-1 真实项目辅导历史页面。会话和用户消息属于业务记录，图的内部工具消息与检查点不是直接展示的对话列表。

当前真实项目页面 / 现有业务数据 / 2026-10-07。只读打开既有辅导会话，展示历史与输入区



图 7-2 校验通过的正常路径。“图已保存”早于“业务已提交”；校验失败不进入提交步骤，接受指针保持不变。

数据层	保存什么	是否作为最终事实
PostgreSQL 学习事实	作答、判分、掌握度、复习任务	学习业务事实来源，可用于重建快照
PostgreSQL 辅导业务表	会话、轮次、用户问题、最终回复、错误	turn 完成状态决定是否进入后续模型历史
Redis 学习快照	当前题、近期结果、弱点和下一步动作	可重建缓存；过期或故障可重新同步
LangGraph 检查点	图状态、消息与执行位置	执行事实；尚需业务校验和接受
接受检查点指针	last_committed_checkpoint_id	限定下一轮起点；与最终回复同事务更新

表 7-1 同样叫“状态”并不代表生命周期、权威性和恢复方式相同。

与第06章的联系

第06章的人工审核中断是独立教学实验。生产辅导当前使用图检查点、执行锁和业务校验恢复；不能因此声称产品已具备 HumanInTheLoop 审批界面。

7.2 四种 ID 与两种状态：先建立对应关系

会话 conversation_id 用于产品中的一个辅导对话；每条请求还带客户端生成的 client_message_id。服务端为执行建立 turn_id，图则使用 graph_thread_id 保存多轮状态。checkpoint_id 是线程内某次图快照的位置。把这些 ID 都叫“消息 ID”会导致重复检测、工具关联和恢复逻辑混淆。

标识或状态	在哪里产生	解决的问题
conversation_id	服务端创建会话	定位产品会话和会话锁
client_message_id	客户端提交请求	同会话重复请求只对应一个 turn
turn_id	reserve 创建业务轮次	关联用户消息、回复、trace 与错误

标识或状态	在哪里产生	解决的问题
graph_thread_id / checkpoint_id	服务层与 LangGraph	定位图线程以及明确的恢复快照
turn.status / draft.status	业务仓储 / 结构化草稿	执行成功与回答证据状态不同

表 7-2 业务唯一键是 conversation_id + client_message_id；ToolMessage 的 tool_call_id 属于单次工具协议，另有用途。

源码节选：轮次唯一约束与请求摘要字段

定位：apps/api/study_agent/infrastructure/tutoring_models.py，第 35-57 行。

```python
class TutorTurnModel(Base):
    __tablename__ = "tutor_turns"
    __table_args__ = (
        UniqueConstraint("conversation_id", "client_message_id", name="uq_tutor_client_message"),
    )
    id: Mapped[UUID] = mapped_column(primary_key=True, default=uuid4)
    conversation_id: Mapped[UUID] = mapped_column(
        ForeignKey("tutor_conversations.id", ondelete="CASCADE"), index=True
    )
    client_message_id: Mapped[UUID] = mapped_column()
    request_hash: Mapped[str] = mapped_column(String(64))
    content: Mapped[str] = mapped_column(Text)
    intent: Mapped[str] = mapped_column(String(30), default="materials")
    status: Mapped[str] = mapped_column(String(20), default="running")
    response: Mapped[dict[str, Any] | None] = mapped_column(JSON)
    usage: Mapped[dict[str, Any] | None] = mapped_column(JSON)
    trace: Mapped[list[dict[str, Any]] | None] = mapped_column(JSON)
    error_code: Mapped[str | None] = mapped_column(String(100))
    error_message: Mapped[str | None] = mapped_column(Text)
    model: Mapped[str | None] = mapped_column(String(100))
    prompt_version: Mapped[str] = mapped_column(String(50), default="tutor-v1")
    started_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), server_default=func.now())
    finished_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
```

TutorTurnModel 是辅导轮次表；UniqueConstraint 阻止同会话同客户端 ID 重复建轮次。

request_hash 保存正文和 intent 的确定性摘要，response、usage 和 trace 保存本轮结果。

status=completed 表示回复已提交；回复内部 answer_status 可以是 insufficient_evidence。

例如一个无资料问题获得“当前知识库没有依据”的回复，业务轮次可为 completed，而草稿状态为 insufficient_evidence，因为系统已正确完成一次无依据回应。反之，格式错误、过期引用或超时会使 turn 失败，未接受内容不能作为后续模型的已完成历史。状态枚举必须按所属对象阅读。

例子中的标识

教学代码使用随机 UUID 和 ch07-demo 线程，仅存在本进程内存。它们不是业务库中的真实会话，也不会改变用户学习记录。

7.3 PostgreSQL Checkpointer：初始化和运行应分开

Checkpointer（检查点保存器）在图运行时保存状态快照。生产使用 AsyncPostgresSaver，通过 psycopg 异步连接池访问 PostgreSQL；业务 ORM 则由 SQLAlchemy 使用自己的 session。二者都是 PostgreSQL，并不意味着共享同一事务或相同表结构。API 启动只探测框架表是否存在，初始化由显式部署命令完成。

源码节选：启动只读探测，缺表则不开放辅导

定位：apps/api/study_agent/infrastructure/checkpointer.py，第 60-79 行。

```
60              await pool.open()
61              # Read-only probe: never implicitly create the framework tables on startup.
62              async with asyncio.timeout(3):
63                  async with pool.connection() as connection:
64                      cursor = await connection.execute(
65                          "SELECT to_regclass('public.checkpoints') AS name"
66                      )
67                      row = await cursor.fetchone()
68                      if not row or not row["name"]:
69                          raise RuntimeError("checkpoint schema not initialized")
70              app.state.tutor_checkpointer = AsyncPostgresSaver(pool)
71      except Exception:
72          app.state.tutor_checkpoint_error = "TUTOR_CHECKPOINT_UNAVAILABLE"
73      try:
74          yield
75      finally:
76          if pool:
77              await pool.close()
78          if lock_engine:
79              await lock_engine.dispose()
```

前置 pool 已按检查点连接参数创建；to_regclass 查询只检测 public.checkpoints 是否存在。

探测成功才把 AsyncPostgresSaver 放到 app.state；失败记录 TUTOR_CHECKPOINT_UNAVAILABLE。

生命周期退出后关闭连接池和独立锁 engine，避免泄漏连接资源。



图 7-3 初始化属于部署动作，日常请求不会偷偷创建框架表。

辅导关闭或保存器不可用时，available 会在模型创建前抛出业务错误；它不会降级成不带检查点的临时对话。运行设置可以为检查点指定单独数据库 URL，默认回退到业务数据库。学习命令应使用已准备好的环境和测试库，避免对自己的真实库随意执行初始化或故障注入。

内存实验的边界

InMemorySaver 只在同一进程内保存状态；重启持久性需要实际 PostgreSQL。当前依赖锁能运行 TutorAnswerDraft 的序列化，但实验出现未来版本允许列表提示，升级时应重新验证自定义类型，不能根据旧锁版本成功推断未来版本行为。

7.4 幂等：同一消息重复发来，应重放什么

Idempotency（幂等）在这里表示同一客户端消息重复提交时复用同一个业务轮次，而不是再次调用模型。request_digest 把 content 和 supplied intent 序列化后取 SHA-256，字典键顺序固定，因此相同输入得到相同摘要。它没有做语义归一化；空格变化、正文变化或意图变化可以成为不同请求内容。

源码节选：请求摘要与服务端意图判断

定位：apps/api/study_agent/application/tutoring.py，第 41-65 行。

```python
def request_digest(content: str, intent: str) -> str:
    return hashlib.sha256(
        json.dumps(
            {"content": content, "intent": intent},
            ensure_ascii=False,
            sort_keys=True,
            separators=(",", ":"),
        ).encode()
    ).hexdigest()


def effective_intent(content: str, supplied: str, anchored: bool) -> str:
    # Deterministic escalation only; client labels never grant permissions or release scope.
    if re.search(r"错题|答错|最近.*错", content):
        return "mistakes"
    if re.search(r"薄弱|掌握|正确率|学习进度|复习.*(计划|顺序|建议)|我的.*(学习|弱点)", content):
        return "progress"
    if anchored and supplied not in {"progress", "mistakes"}:
        return "answered_question"
    if supplied == "general" and not re.fullmatch(
        r"[\s!! ?? 。 , ]*(你好|您好|嗨|hello|hi|谢谢|你能做什么)[\s!! ?? 。 , ]*", content, re.I
    ):
        return "materials"
    return supplied
```

request_digest 的输入是正文和客户端 intent；sort_keys 和固定 separators 保证确定性序列化。

摘要用来识别同 ID 请求是否冲突，effective_intent 用来升级必需事实读取，二者职责不同。

客户端把“最近错题”标成 materials 时，执行意图仍会被确定性规则提升为 mistakes。

重复情况	处理方式	应观察的结果
同 ID、同内容，已完成	返回已存 response	idempotent_replay=True；不重复执行图
同 ID、不同正文或 intent	ensure_hash 拒绝	TUTOR_MESSAGE_CONFLICT，HTTP 409
同 ID，已有执行者运行	短暂查询已预留 turn	返回现有状态，不创建第二轮
新 ID，同会话正在处理另一请求	无法取得会话锁	TUTOR_THREAD_BUSY，HTTP 409
失败后再次使用同 ID	重放失败终态	显式重试要使用新的客户端消息 ID

表 7-3 重新发送与显式重试不同；不要把网络超时自动换新 ID，否则可能触发额外模型执行。

submit 在获取锁前先查已有轮次，锁内再次查询，处理查询与锁之间的时间窗口。未取得锁时，会短暂等待拥有者把预留行提交为可见，再返回已有结果；这些查询不运行图。如果仍找不到对应轮次，就告诉客户端会话忙。唯一约束和会话锁相互配合，不能只靠前端禁用发送按钮。

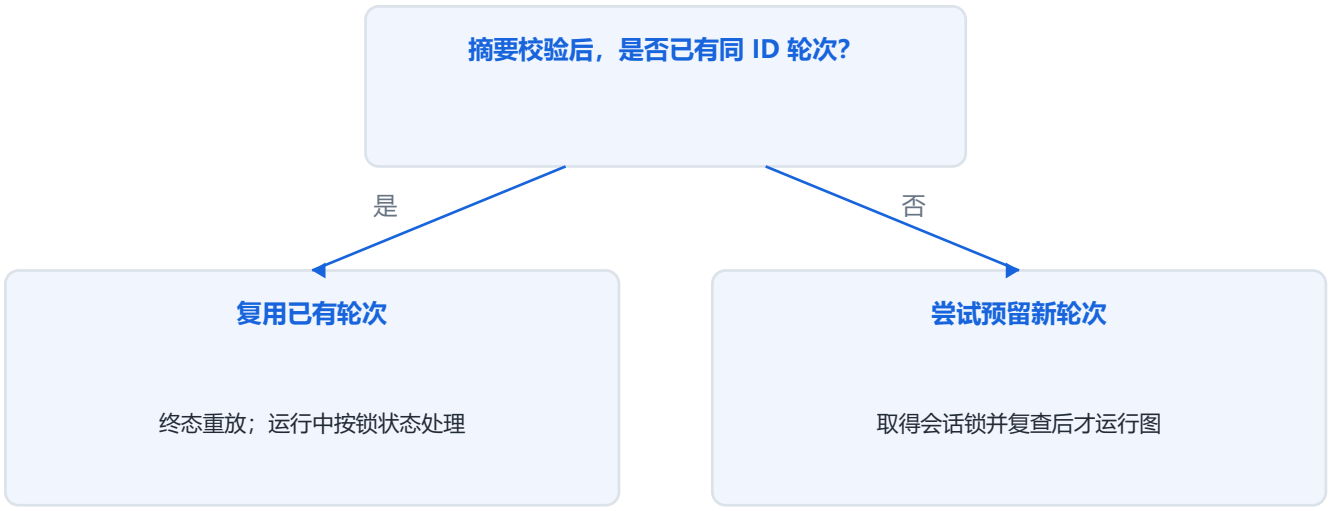


图 7-4 同 ID 先核对摘要；摘要不同立即返回冲突，不进入这里的两条路径。新请求仍需通过会话状态和锁检查，会话忙时不能预留。

7.5 执行锁与短事务：等待模型时不持有长业务事务

TutorRepository.execution_lock 使用 PostgreSQL 会话级 advisory lock（咨询锁）。它从 conversation_id 的前 8 字节构造有符号整数键，通过 pg_try_advisory_lock 立即尝试获取；释放在 finally 中完成。锁连接采用 AUTOCOMMIT，用于让同一会话只由一个执行者推进，而不是把整个模型调用包进数据库事务。

源码节选：一条连接持有会话执行锁

定位：apps/api/study_agent/infrastructure/tutoring.py，第 26-46 行。

```python
    @asynccontextmanager
    async def execution_lock(self, conversation_id: UUID) -> AsyncIterator[bool]:
        key = int.from_bytes(conversation_id.bytes[:8], "big", signed=True)
        try:
            connection = await self.engine.connect()
        except PoolTimeout as exc:
            raise DomainError(
                "TUTOR_CONCURRENCY_LIMIT", "辅导并发达到上限，请稍后重试", status_code=503
            ) from exc
        try:
            connection = await connection.execution_options(isolation_level="AUTOCOMMIT")
            acquired = await connection.scalar(
                text("SELECT pg_try_advisory_lock(:key)"), {"key": key}
            )
            try:
                yield bool(acquired)
            finally:
                if acquired:
                    await connection.execute(text("SELECT pg_advisory_unlock(:key)"), {"key": key})
        finally:
            await connection.close()
```

这是异步上下文管理器，调用者使用 async with ... as acquired 获取是否成功。

advisory lock 是会话级，connection 必须保持占用到执行结束并明确 unlock。

连接池取连接超时映射为 TUTOR_CONCURRENCY_LIMIT；finally 保证关闭连接。

reserve 使用短 session 提交 turn 和用户消息，随后模型与工具在事务之外运行。complete 再开启短 session，把最终回复、轮次完成状态、usage、trace 和接受指针一起提交。失败只更新轮次错误和终态。独立锁 engine 默认 pool_size=4、max_overflow=0，不同会话可以并发，但系统并发仍受池容量约束。

阶段	数据库作用	事务范围
execution_lock	为整个会话执行持有咨询锁	专用连接 AUTOCOMMIT；不是长业务事务
reserve	预留轮次并记录用户问题	短事务提交后才能执行图
图与外部模型	推进图并产生检查点	独立于业务提交；可能产生未接受状态
complete	回复、完成状态和接受指针	同一个短业务事务原子提交
fail	错误、用量、轨迹与失败终态	不推进业务接受指针

表 7-4 咨询锁负责执行互斥，事务负责业务写入原子性；图保存器另有持久化过程。

原子性的范围

“回复与接受指针一起提交”指 complete 内的业务事务。检查点在此前由图保存器写入，并未参与同一个分布式事务。因此可能有多余快照，但业务不会接受未经最终校验的输出。



图 7-5 正式回复、轮次状态和接受指针由短业务事务一起提交；图保存器另行写入，指针选择已接受的快照。

7.6 从接受指针启动：Overwrite 为什么不可少

last_committed_checkpoint_id 保存最近一次业务已经接受的图快照。下一轮 submit 将 thread_id 与该 checkpoint_id 放入 RunnableConfig，明确选择起点；历史消息从 committed_history 读取，再加本轮用户问题。程序不能简单认为“不传 checkpoint_id 读到的最新快照一定可信”。

源码节选：指定接受快照并重建本轮状态

定位：apps/api/study_agent/application/tutoring.py，第 342-371 行。

```
342                 config: RunnableConfig = {
343                     "configurable": {
344                         "thread_id": conversation.graph_thread_id,
345                         "checkpoint_id": conversation.last_committed_checkpoint_id,
346                     },
347                     "recursion_limit": 32,
348                 }
349                 async with asyncio.timeout(self.settings.tutor_timeout_seconds):
350                     state = await agent.ainvoke(
351                         {
352                             "messages": Overwrite(messages),
353                             "evidence": Overwrite({}),
354                             "evidence_flags": Overwrite([]),
```

代码续

```
                            "structured_response": None,
                            "turn_id": str(turn.id),
                        },
                        config,
                        context=context,
                    )
                    response = await self.final_response(conversation, turn, state, budget.usage())
                    snapshot = await agent.aget_state(
                        {"configurable": {"thread_id": conversation.graph_thread_id}}
                    )
                    await self.repository.complete(
                        conversation_id,
                        turn.id,
                        response,
                        snapshot.config["configurable"]["checkpoint_id"],
                        budget.trace,
                    )
```

messages 已由完成轮次的业务历史转换成 HumanMessage / AIMessage，并加入当前问题。

Overwrite 绕过累积 reducer，替换 messages、evidence 和 evidence_flags；structured_response 清为 None。

ainvoke 完成后调用 final_response，再把最新图 checkpoint_id 和业务响应传给 complete。

如果 evidence 只传空字典但仍按 merge_evidence 合并，旧账本可能继续保留；如果 messages 只追加，则未接受内容也可能叠入历史。Overwrite 表达明确替换，确保上一轮草稿和证据不冒充本轮结果。turn_id 随新业务轮次更新，最终恢复也据此检查快照是否真的属于目标轮次。

源码节选：完成回复与接受指针同事务更新

定位：apps/api/study_agent/infrastructure/tutoring.py，第 168-193 行。

```
        # The accepted graph pointer and final business response share one short transaction.
        async with self.sessions() as session:
            conversation = await session.get(TutorConversationModel, conversation_id)
            turn = await session.get(TutorTurnModel, turn_id)
            if conversation is None or turn is None:
                raise DomainError("TUTOR_TURN_NOT_FOUND", "辅导轮次不存在", status_code=404)
            if turn.status == "completed":
                return
            turn.status = "completed"
            turn.response = response
            turn.usage = response["usage"]
            turn.trace = trace
            turn.finished_at = datetime.now(UTC)
            conversation.last_committed_checkpoint_id = checkpoint_id
            session.add(
                TutorMessageModel(
                    id=uuid4(),
                    conversation_id=conversation_id,
```

代码续

```
                    turn_id=turn_id,
                    sequence=await self._next_sequence(session, conversation_id),
                    role="assistant",
                    content=response["message"],
                    citations=response["citations"],
                )
            )
            await session.commit()
```

conversation 与 turn 在同一个 session 中读取，已完成 turn 直接返回，避免重复助手消息。

turn.response、usage、trace 和 conversation.last_committed_checkpoint_id 一起更新。

新增 assistant 消息后 session.commit，使回复和接受指针具有同一业务提交边界。

committed_history 联结 TutorTurnModel，并只保留 completed 轮次的消息。失败轮次的用户问题仍可能保留在业务审计消息列表中；后续模型历史会过滤它。页面看到记录、数据库有记录和模型接受的历史是三个不同问题，排查时必须分别观察。

7.7 中断恢复：仅补交完整且重新验证过的结果

recover_locked 只在取得排他会话锁后调用，说明当前没有另一个执行者仍拥有该轮。它读取线程最新快照；只有 snapshot.next 为空、快照 turn_id 匹配目标 turn，并有 structured_response，才把草稿交给 final_response 再次校验。这里恢复的是“图已完成、业务尚未提交”的窗口，不是无条件重跑外部模型。

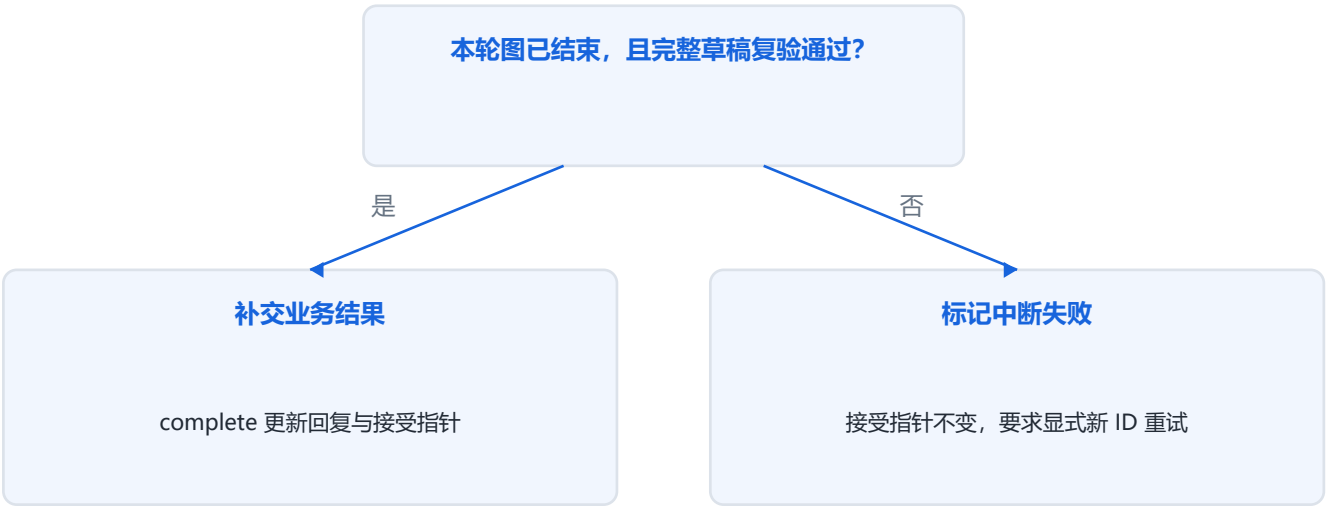


图 7-6 仅在取得排他会话锁后检查最新快照。还必须匹配 turn_id；缺草稿、未结束或复验失败都走“否”，不会自动重跑模型。

故障窗口	快照情况	当前恢复策略
预留后、图还未完成	无匹配的完整草稿	标记 TUTOR_INTERRUPTED，不自动重跑模型
图完成后、业务提交前	匹配 turn，无下一节点，有草稿	final_response 复验通过后补交
草稿存在但资料已失效	引用复验失败	标记失败；显式使用新 ID 重试

故障窗口	快照情况	当前恢复策略
回复和接受指针已提交	turn 已 completed	重复请求重放已存业务响应
图版本不兼容或接受指针缺失	不能确定安全起点	TUTOR_CONTEXT_UNAVAILABLE，要求新会话

表 7-5 以当前 TutorService 为准，不能把检查点存在等同于支持所有节点透明恢复。

恢复后无法准确还原所有供应商用量，因此响应将 known=False、total_tokens=None，并标记 recovered。取消处理使用 asyncio.shield 尝试保存 cancelled 终态；异常映射把供应商原始错误转为业务代码。客户端收到失败后仍应明确重试，而不是后台默默追加另一次付费执行。

已有验收与本章实验

2026-10-07 学习准备记录中有 19 项 PostgreSQL 验收，覆盖并发同 ID、故障窗口、失败草稿隔离和资料失效。这是历史执行证据。本章制作时再次运行的是下面的内存教学实验，不能将两者合并称为本次完整数据库复验。

7.8 Redis Snapshot：快速恢复学习界面，不接管图历史

RedisLearningSessionSnapshotStore 的键为 study:session:{session_id}:state，值是 JSON 字典并带 TTL。它缓存学习会话当前题、已选题、已答题、最近五次结果、弱点和下一步动作。StudyService._sync_runtime_state 从 UnitOfWork 的题目分配、作答记录和掌握度重新组织这些字段，因此 Redis 不是唯一事实来源。

源码节选：从业务事实构建学习快照

定位：apps/api/study_agent/application/study.py，第 512-542 行。

```
512         assignments = await uow.study_sessions.list_questions(study_session.id)
513         answers = list(await uow.answer_records.list_for_session(study_session.id))
514         mastery = await uow.mastery.list(study_session.knowledge_base_id)
515         state: dict[str, object] = {
516             "session_id": str(study_session.id),
517             "mode": study_session.mode,
518             "status": study_session.status,
519             "current_question_id": str(current_question.id) if current_question else None,
520             "selected_question_ids": [str(item.question_id) for item in assignments],
521             "answered_question_ids": [str(item.question_id) for item in answers],
522             "recent_results": [
523                 {
524                     "question_id": str(item.question_id),
525                     "knowledge_point_id": str(item.knowledge_point_id),
526                     "verdict": item.verdict,
527                     "score": item.score,
528                     "max_score": item.max_score,
529                 }
530                 for item in answers[-5:]
531             ],
532             "weak_point_ids": [
533                 str(item.knowledge_point_id) for item in mastery if item.mastery_score < 0.5
```

代码续

```
            ],
            "remaining_question_count": max(
                0,
                study_session.planned_question_count - study_session.answered_question_count,
            ),
            "next_action": "await_answer" if current_question else "finish",
            "updated_at": datetime.now(UTC).isoformat(),
        }
        await state_store.save(study_session.id, state)
```

uow 提供 PostgreSQL 业务仓储；study_session 和 current_question 表示当前业务学习会话。

调用者已确认存在 state_store；此段读取分配、作答和掌握度，构造 JSON 友好字段。

快照还保存弱点、剩余数量、next_action 和更新时间；最后使用 state_store.save 保存。

Redis get 捕获连接错误或 JSON 解析失败并返回 None，save 和 delete 捕获 RedisError 后返回。设计允许缓存短暂不可用，不应因此丢掉已提交作答。下一次业务动作可以重新同步快照；具体界面恢复仍取决于调用路径，不能只看类名就声称所有场景都会自动恢复。

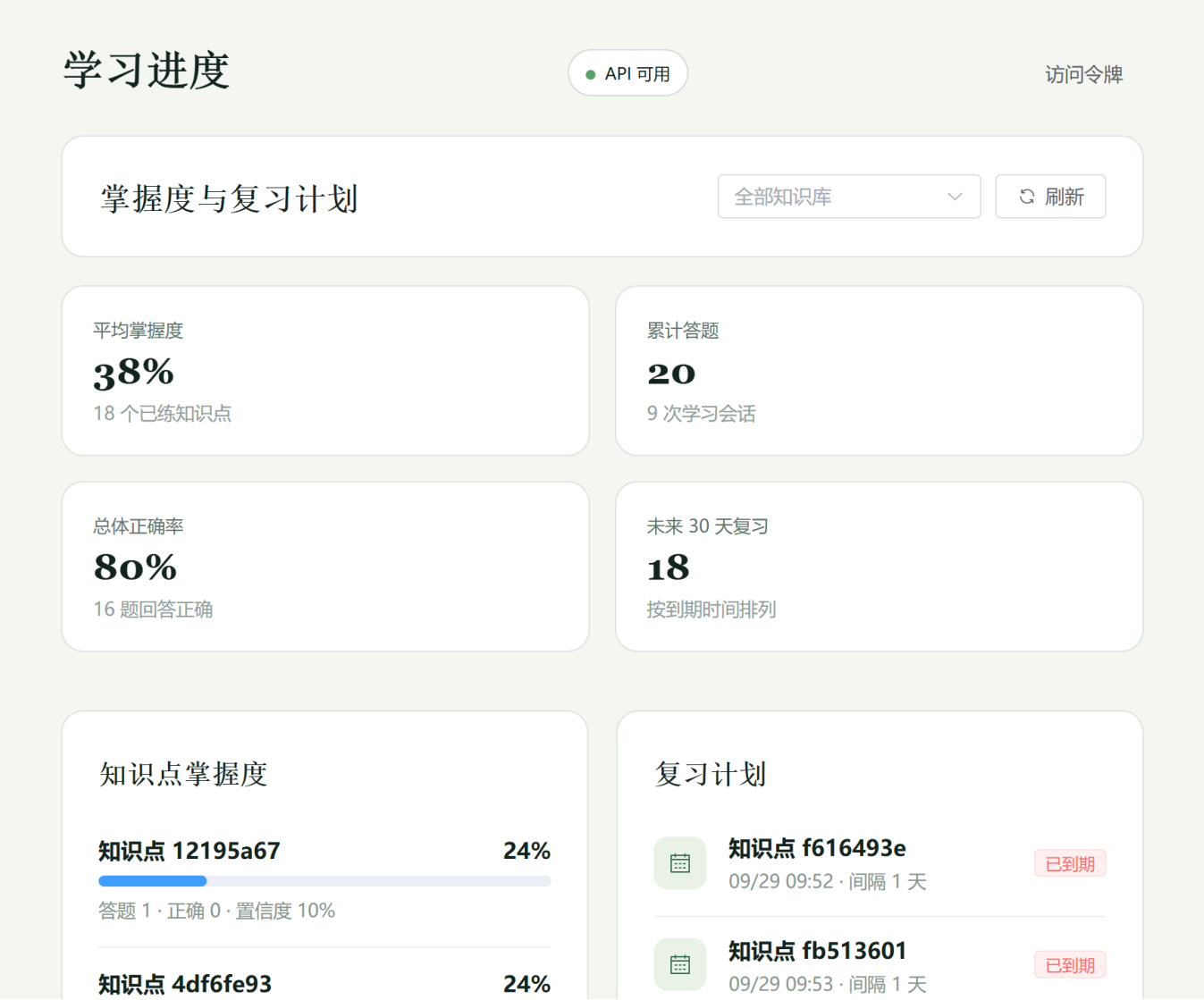


图 7-7 真实项目学习进度页面。正确率和掌握度来自业务数据与确定性计算；Redis 快照用于运行信息缓存，LangGraph 检查点用于辅导图执行。

当前真实项目页面 / 现有业务数据 / 2026-10-07。当前业务记录的掌握度与复习计划，不是模型质量评测

术语对齐

旧 RedisStudyStateStore 是兼容别名，新教学名称为 RedisLearningSessionSnapshotStore。这里的 Snapshot 不是 StateSnapshot，业务 Protocol 也不是 LangGraph Store；它们不应在 PDF 中统一简称为“记忆”。

7.9 完整离线实验：最新快照与接受快照不同

下面代码在项目根目录运行，加载第05章固定模型与查询 fixture，再创建真实辅导图和 InMemorySaver。首次问答结束后保留 accepted.config，然后人为写入一份未接受问题和证据。第二轮明确从 accepted.config 开始，并按生产方式替换历史与本轮证据。人为写快照只是模拟故障窗口，不执行真实业务事务。

完整可运行实验：接受起点与未接受状态隔离

```python
"""教学简化实验：真实 Agent 图的接受检查点与未接受快照。

从 D:/Study/StudyAgent 根目录运行；离线、不写业务数据库。
"""
import asyncio
import runpy
import time
from pathlib import Path
from uuid import UUID, uuid4

from langchain_core.messages import HumanMessage
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Overwrite
from study_agent.agents.tutor.agent import create_tutor_agent
from study_agent.agents.tutor.context import TutorContext
from study_agent.agents.tutor.middleware import RunBudget
from study_agent.application.tutoring import request_digest

fixture = runpy.run_path(str(Path.cwd() / "examples/learning/agent_tool_loop.py"))

async def main():
    kb = UUID(int=1)
    model = fixture["ScriptedModel"]()
    graph = create_tutor_agent(
        model, fixture["ScopedQueries"](kb), fixture["FixedRetrieval"](kb, False),
        RunBudget(time.monotonic() + 15), InMemorySaver(),
    )
    config = {"configurable": {"thread_id": "ch07-demo"}}
    context = TutorContext(uuid4(), uuid4(), kb)
    first = await graph.ainvoke(
        {"messages": [HumanMessage(content="TCP 是什么？")], "turn_id": str(context.turn_id),
         "evidence": {}, "evidence_flags": []}, config, context=context,
    )
    accepted = await graph.aget_state(config)
    await graph.aupdate_state(
        accepted.config,
        {"messages": Overwrite([HumanMessage(content="未接受问题")]),
         "evidence": Overwrite({"unaccepted": {"quote": "未接受证据"}})},
        as_node="model",
    )
    latest = await graph.aget_state(config)
    context2 = TutorContext(context.conversation_id, uuid4(), kb)
    second = await graph.ainvoke(
        {"messages": Overwrite([HumanMessage(content="已提交历史"), HumanMessage(content="再问 TCP")]),
         "evidence": Overwrite({}), "evidence_flags": Overwrite([]),
         "structured_response": None, "turn_id": str(context2.turn_id)},
        accepted.config, context=context2,
    )
    input_messages = model.calls[2]
    base_digest = request_digest("TCP", "materials")
    same_digest = request_digest("TCP", "materials")
    changed_digest = request_digest("TCP", "progress")
    print("same_request_hash:", base_digest == same_digest)
    print("changed_intent_hash:", base_digest == changed_digest)
```

代码续

```
    print("first_status:", first["structured_response"].status)
    contains_latest = any(m.content == "未接受问题" for m in latest.values["messages"])
    print("latest_contains_unaccepted:", contains_latest)
    contains_next = any(m.content == "未接受问题" for m in input_messages)
    print("next_input_contains_unaccepted:", contains_next)
    print("new_turn_evidence_ids:", sorted(second["evidence"]))

asyncio.run(main())
```

完整代码依赖仓库 examples/learning/agent_tool_loop.py；使用项目 .venv，无供应商请求和数据库写入。

aupdate_state 在本实验用于人为形成未接受快照；生产恢复条件应阅读 recover_locked。

第二轮从 accepted.config 启动，Overwrite 清空证据；打印进入第二轮首次模型调用的消息是否含未接受问题。

结果解释：相同正文与 intent 的摘要相同；改 intent 后摘要不同。最新快照含未接受问题，但第二轮模型输入不含它；新轮账本只有 fixture:tcp:1。

本次实际输出为 same_request_hash: True、changed_intent_hash: False、first_status: answered、latest_contains_unaccepted: True、next_input_contains_unaccepted: False。实验还输出 new_turn_evidence_ids: ['fixture:tcp:1']，说明人工写入的 unaccepted 证据没有被当成本轮依据。

• 练习一：去掉 evidence 的 Overwrite，使用空字典更新，观察 reducer 与旧证据之间的关系。

• 练习二：阅读 committed_history 的 join 与 completed 条件，解释失败用户消息为何可审计但不入下一轮模型历史。

• 练习三：阅读 PostgreSQL test_r06_orphan_fault_windows_recover_without_provider_repeat，画出三个故障窗口。

参考思路

第一题必须区分选择的接受快照中已有证据与未接受分支证据；第二题区分存储记录和模型输入；第三题把图完整草稿、最终引用校验与业务提交排列在同一时间线上。内存实验不证明 PostgreSQL 锁、唯一约束或进程重启持久性。

7.10 综合练习与参考思路

练习 1：图最新检查点存在，为什么仍不能直接作为下一轮起点?

参考思路：它可能包含图已运行但业务未接受的草稿。下一轮从 last_committed_checkpoint_id 开始，重建 completed 历史并清空本轮证据。

练习 2：业务回复和检查点是否在同一个事务里?

参考思路：图保存器先写检查点，complete 在一个短业务事务内更新回复、turn 完成状态与接受指针。两者没有组成单一分布式事务。

练习 3：失败之后重复同 client_message_id 会发生什么？

参考思路：同正文和 intent 重放已保存失败终态；改内容则冲突。显式新尝试要用新 ID，普通网络重发保留旧 ID。

练习 4：为什么 Redis 丢失不等于学习事实丢失？

参考思路：Redis 保存可重建业务快照，作答、分配和掌握度由业务数据库保存。它也不承担 LangGraph 的图持久化。

7.11 源码阅读清单与下一步

文件或入口	关注的函数 / 类	阅读时回答的问题
apps/api/study_agent/application/tutoring.py	request_digest / TutorService.submit / recover_locked	重复消息、接受起点与恢复条件如何衔接
apps/api/study_agent/infrastructure/tutoring.py	execution_lock / reserve / complete / committed_history	执行互斥和短事务分别保护什么
apps/api/study_agent/infrastructure/tutoring_models.py	TutorConversationModel / TutorTurnModel / TutorMessageModel	业务 ID、唯一约束与接受指针在哪里保存
apps/api/study_agent/infrastructure/checkpointer.py	open_tutor_checkpointer / setup_checkpointer	为什么初始化和运行资源分开
apps/api/study_agent/infrastructure/study_state.py	RedisLearningSessionSnapshotStore	业务缓存如何容忍故障和过期
apps/api/study_agent/application/study.py	StudyService._sync_runtime_state	快照从哪些业务事实重建
apps/api/tests/test_tutor_postgres_acceptance.py	test_r03 / test_r06 / test_r08 / test_r13	并发幂等、故障恢复和草稿隔离有什么数据库验收
evaluation/results/2026-10-07-学习准备验证.json	学习准备验收记录	历史数据库和供应商验收的实际范围

下一章：第08章回到确定性学习业务：判分、掌握度、选题与复习，并讨论如何审阅 Agent 与 RAG 的质量证据。

主要参考：本地飞书 LangChain、Agent、RAG 与 LangGraph 课程的教学组织方式，以及当前项目源码。具体框架接口以锁定环境与官方文档为准。